

# Windows Notification System - Comprehensive Technical Reference

---

## Table of Contents

1. [Overview](#)
  2. [How Windows Notifications Work](#)
  3. [Toast Notification Framework](#)
  4. [Multiple Notifications from Same Application](#)
  5. [Grouping and Threading](#)
  6. [Silencing Notifications](#)
  7. [Notification Persistence and Loss Prevention](#)
  8. [Customizing the Toast UI](#)
  9. [Security Model](#)
  10. [Multi-Machine Considerations](#)
  11. [PowerShell Implementation Guide](#)
  12. [Advanced Features](#)
- 

## Overview

Windows 10 and Windows 11 use the **Windows Runtime (WinRT) Toast Notification API** for modern app notifications. This system replaced the older balloon tip notifications and provides:

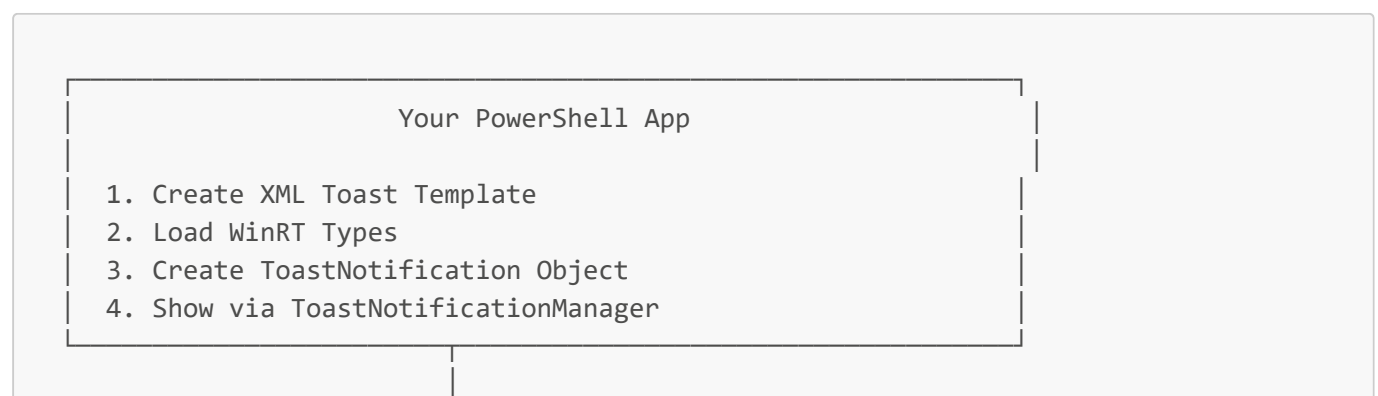
- Rich, interactive notifications with buttons, images, progress bars
- Notification grouping and threading
- Persistent notification history in Action Center
- User-controlled Do Not Disturb (Focus Assist)
- Per-app notification settings and permissions
- Cross-device notification sync (optional)

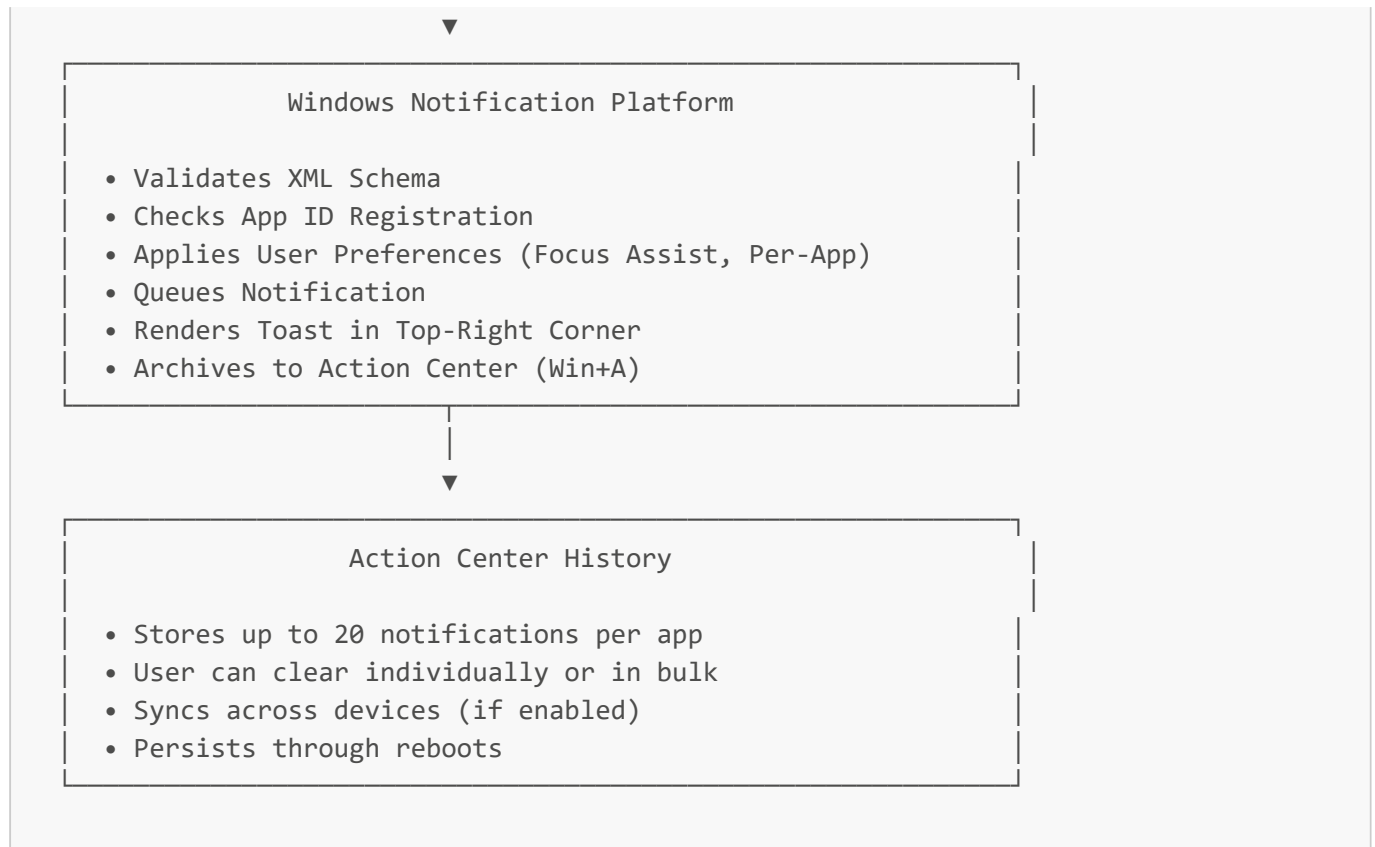
**Key Namespace:** `Windows.UI.Notifications`

---

## How Windows Notifications Work

### Architecture





## Process Flow

1. **App ID Registration:** When you call `CreateToastNotifier($APP_ID)`, Windows creates or uses an existing registration for that App ID in the registry.
2. **XML Template:** You define the notification content in XML format following the [Toast Content Schema](#).
3. **WinRT Types:** PowerShell loads the Windows Runtime types from `Windows.UI.Notifications.dll` which ships with Windows 10/11.
4. **Show():** Calling `.Show()` submits the notification to the Windows notification broker service.
5. **Rendering:** Windows displays the toast in the lower-right corner for ~5 seconds (default), then moves it to Action Center.

---

## Toast Notification Framework

### Basic Template Types

Windows provides several built-in toast templates:

| Template                 | Description                 | Fields                              |
|--------------------------|-----------------------------|-------------------------------------|
| <code>ToastText01</code> | Single line of text         | Title only                          |
| <code>ToastText02</code> | Two lines of text           | Title + Message                     |
| <code>ToastText03</code> | Two lines with wrapped text | Title + Long Message                |
| <code>ToastText04</code> | Three lines                 | Title + Message + Secondary Message |

| Template                         | Description          | Fields                              |
|----------------------------------|----------------------|-------------------------------------|
| <code>ToastImageAndText01</code> | Image + one line     | Image + Title                       |
| <code>ToastImageAndText02</code> | Image + two lines    | Image + Title + Message             |
| <code>ToastImageAndText03</code> | Image + wrapped text | Image + Title + Long Message        |
| <code>ToastImageAndText04</code> | Image + three lines  | Image + Title + Message + Secondary |

## Modern Adaptive Templates

Windows 10+ supports **adaptive toast templates** with richer layouts:

```
<toast>
  <visual>
    <binding template="ToastGeneric">
      <text>Title Text</text>
      <text>Body Text Line 1</text>
      <text>Body Text Line 2</text>
      <image placement="appLogoOverride" src="C:\path\to\icon.png"/>
      <image placement="hero" src="C:\path\to\banner.png"/>
    </binding>
  </visual>
  <actions>
    <action content="Reply" arguments="reply" />
    <action content="Dismiss" arguments="dismiss" />
  </actions>
  <audio src="ms-winsoundevent:Notification.Default" loop="false" />
</toast>
```

### Key Elements:

- `<visual>`: Display content (text, images)
- `<actions>`: Interactive buttons (up to 5)
- `<audio>`: Sound file or Windows system sound
- `<header>`: For grouping notifications

## Multiple Notifications from Same Application

### How Windows Handles Multiple Toasts

When you show multiple notifications from the same App ID:

1. **Queuing**: Windows queues notifications if more than 3-4 are shown rapidly
2. **Display Limit**: Maximum 3 toasts visible simultaneously on screen
3. **Action Center**: All notifications stored in history (up to 20 per app)
4. **Replacement**: New notifications push older ones to Action Center faster

### Example: Sending Multiple Notifications

```

function Show-WindowsNotification {
    param(
        [string]$Title,
        [string]$Message,
        [string]$Tag = $null,      # Unique identifier for replacement
        [string]$Group = $null    # Group identifier for threading
    )

    [Windows.UI.Notifications.ToastNotificationManager, Windows.UI.Notifications,
    ContentType = WindowsRuntime] | Out-Null
    [Windows.UI.Notifications.ToastNotification, Windows.UI.Notifications,
    ContentType = WindowsRuntime] | Out-Null
    [Windows.Data.Xml.Dom.XmlDocument, Windows.Data.Xml.Dom.XmlDocument,
    ContentType = WindowsRuntime] | Out-Null

    $APP_ID = "CreateEmailEventAndTeamsChat"
    $template = @"
<toast>
  <visual>
    <binding template="ToastText02">
      <text id="1">$Title</text>
      <text id="2">$Message</text>
    </binding>
  </visual>
</toast>
"@

    $xml = New-Object Windows.Data.Xml.Dom.XmlDocument
    $xml.LoadXml($template)
    $toast = New-Object Windows.UI.Notifications.ToastNotification $xml

    # Set Tag and Group for replacement/threading
    if ($Tag) { $toast.Tag = $Tag }
    if ($Group) { $toast.Group = $Group }

    [Windows.UI.Notifications.ToastNotificationManager]::CreateToastNotifier($APP_ID).
    Show($toast)
}

# Send multiple notifications
Show-WindowsNotification -Title "Email Created" -Message "Email to
john.doe@irs.gov" -Tag "email-001"
Show-WindowsNotification -Title "Event Created" -Message "Meeting scheduled for
2PM" -Tag "event-001"
Show-WindowsNotification -Title "Teams Chat" -Message "Opened chat with Jane" -Tag
"teams-001"

```

## Notification Throttling Best Practice

To avoid overwhelming users, implement throttling:

```
$script:lastNotificationTime = $null
$script:notificationQueue = @()

function Queue-Notification {
    param([string]$Title, [string]$Message)

    $script:notificationQueue += @{ Title = $Title; Message = $Message; Time =
    Get-Date }

    # Only show one notification per 3 seconds
    $now = Get-Date
    if (-not $script:lastNotificationTime -or
        ($now - $script:lastNotificationTime).TotalSeconds -ge 3) {

        $notification = $script:notificationQueue[0]
        Show-WindowsNotification -Title $notification.Title -Message
$notification.Message
        $script:lastNotificationTime = $now
        $script:notificationQueue = $script:notificationQueue | Select-Object -
Skip 1
    }
}
```

---

## Grouping and Threading

### Tag and Group System

Windows supports **Tag** and **Group** properties for notification management:

- **Tag**: Unique identifier within a group. Used to replace or remove specific notifications.
- **Group**: Category identifier. Used to group related notifications together.

### Example: Grouped Notifications

```
# Emails group
Show-WindowsNotification -Title "New Email" -Message "From: Alice" -Tag "email-1" -
-Group "emails"
Show-WindowsNotification -Title "New Email" -Message "From: Bob" -Tag "email-2" -
Group "emails"

# Events group
Show-WindowsNotification -Title "Meeting Soon" -Message "Team standup in 10 min" -
Tag "event-1" -Group "events"
Show-WindowsNotification -Title "Meeting Soon" -Message "1:1 with manager in 30
min" -Tag "event-2" -Group "events"
```

**Result:** Action Center shows these as two separate groups:

- "emails" (2 notifications)
- "events" (2 notifications)

## Header-Based Grouping

Use `<header>` element for visual grouping with a title:

```
<toast>
  <header id="email-header"
    title="New Emails"
    arguments="group:emails"
    activationType="background"/>
  <visual>
    <binding template="ToastText02">
      <text>Email from Alice</text>
      <text>RE: Project Update</text>
    </binding>
  </visual>
</toast>
```

## Replacing Notifications

To update an existing notification, use the same **Tag** and **Group**:

```
# Initial notification
Show-WindowsNotification -Title "Download Started" -Message "0% complete" -Tag
"download-1" -Group "downloads"

# Update (replaces previous)
Show-WindowsNotification -Title "Download Progress" -Message "50% complete" -Tag
"download-1" -Group "downloads"

# Final update (replaces again)
Show-WindowsNotification -Title "Download Complete" -Message "File saved" -Tag
"download-1" -Group "downloads"
```

Only one notification appears in Action Center (the latest one).

---

## Silencing Notifications

### User-Level Controls

Windows provides several ways for users to control notifications:

#### 1. Focus Assist (Do Not Disturb)

**Location:** Settings → System → Focus Assist

**Modes:**

- **Off:** All notifications show
- **Priority only:** Only starred apps/contacts notify
- **Alarms only:** Only alarms and timers
- **Automatic rules:** Schedule-based (e.g., during presentations, gaming)

**Your App Cannot Override:** If Focus Assist blocks your app, notifications are suppressed silently and NOT stored in Action Center.

**2. Per-App Notification Settings**

**Location:** Settings → System → Notifications & actions → [Your App Name]

**User can disable:**

- Show notifications in Action Center
- Show notification banners
- Play notification sounds
- Show notification on lock screen
- Number of visible notifications in Action Center (1-20)

**Registry Location:**

```
HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Notifications\Settings  
\[APP_ID]
```

**Programmatic Silencing****Silent Notifications (No Banner)**

Show notification in Action Center only (no pop-up):

```
<toast launch="silent">  
  <visual>  
    <binding template="ToastText02">  
      <text>Silent Update</text>  
      <text>Check Action Center for details</text>  
    </binding>  
  </visual>  
  <audio silent="true"/>  
</toast>
```

**Suppress Sound**

```
<toast>
  <visual>...</visual>
  <audio silent="true"/>
</toast>
```

## Remove Notification Programmatically

```
function Remove-WindowsNotification {
    param(
        [string]$Tag,
        [string]$Group = ""
    )

    $APP_ID = "CreateEmailEventAndTeamsChat"
    $notifier =
[Windows.UI.Notifications.ToastNotificationManager]::CreateToastNotifier($APP_ID)
    $history = [Windows.UI.Notifications.ToastNotificationManager]::History

    if ($Tag) {
        $history.Remove($Tag, $Group, $APP_ID)
    }
}

# Remove specific notification
Remove-WindowsNotification -Tag "email-001" -Group "emails"
```

## Clear All Notifications for App

```
function Clear-AllWindowsNotifications {
    $APP_ID = "CreateEmailEventAndTeamsChat"
    $history = [Windows.UI.Notifications.ToastNotificationManager]::History
    $history.Clear($APP_ID)
}
```

---

## Notification Persistence and Loss Prevention

### Do Notifications Get Lost?

**Short Answer:** Notifications can be lost in specific scenarios, but Windows has built-in persistence.

### When Notifications Persist

✅ **Notifications ARE stored** in these cases:

- User is away when notification shows → Stored in Action Center

- User dismisses notification → Stored in Action Center (unless manually cleared)
- System reboots → Notifications survive reboot (Action Center history persists)
- Screen is locked → Notification shown on lock screen (if enabled) + Action Center
- Focus Assist enabled → **Depends on mode** (see below)

## When Notifications Are Lost

✗ **Notifications ARE NOT stored** in these cases:

- **Focus Assist "Alarms only" mode:** Notification suppressed completely, not saved
- **App notifications disabled in Settings:** Silently dropped
- **Action Center full (20 notification limit):** Oldest notification removed when 21st arrives
- **User clears Action Center:** Manually removed
- **App unregistered/removed:** All associated notifications cleared
- **Notification XML is malformed:** Fails silently, not shown or stored

## Ensuring Notifications Don't Get Lost

### 1. Set Expiration Time

```
$toast = New-Object Windows.UI.Notifications.ToastNotification $xml
$toast.ExpirationTime = (Get-Date).AddHours(24) # Keep for 24 hours
```

Default expiration: 5 days. Max: 7 days.

### 2. Use Scheduled Notifications

Deliver notification at specific time:

```
$scheduledTime = (Get-Date).AddMinutes(30)
$scheduledToast = New-Object
Windows.UI.Notifications.ScheduledToastNotification($xml, $scheduledTime)
[Windows.UI.Notifications.ToastNotificationManager]::CreateToastNotifier($APP_ID).
AddToSchedule($scheduledToast)
```

### 3. Check Notification History

Query existing notifications:

```
function Get-WindowsNotificationHistory {
    param([string]$AppId = "CreateEmailEventAndTeamsChat")

    $history = [Windows.UI.Notifications.ToastNotificationManager]::History
    $notifications = $history.GetHistory($AppId)

    foreach ($notification in $notifications) {
```

```

[PSCustomObject]@{
    Tag = $notification.Tag
    Group = $notification.Group
    Content = $notification.Content.GetXml()
}
}
}

```

#### 4. Logging and Redundancy

Maintain your own log of sent notifications:

```

function Show-WindowsNotification-WithLogging {
    param([string]$Title, [string]$Message, [string]$Tag)

    try {
        # Show notification
        Show-WindowsNotification -Title $Title -Message $Message -Tag $Tag

        # Log to file
        $logEntry = [PSCustomObject]@{
            Timestamp = Get-Date -Format "yyyy-MM-dd HH:mm:ss"
            Tag = $Tag
            Title = $Title
            Message = $Message
            Status = "Sent"
        }
        $logEntry | Export-Csv -Path ".\NotificationLog.csv" -Append -
        NoTypeInfoInformation
    }
    catch {
        # Log failure
        $logEntry = [PSCustomObject]@{
            Timestamp = Get-Date -Format "yyyy-MM-dd HH:mm:ss"
            Tag = $Tag
            Title = $Title
            Message = $Message
            Status = "Failed: $($_.Exception.Message)"
        }
        $logEntry | Export-Csv -Path ".\NotificationLog.csv" -Append -
        NoTypeInfoInformation
    }
}

```

## Customizing the Toast UI

### Toast Display Duration

Default: **7 seconds** on screen, then moves to Action Center

**Short Duration** (5 seconds):

```
<toast duration="short">...</toast>
```

**Long Duration** (25 seconds):

```
<toast duration="long">...</toast>
```

## Adding Images

**App Logo Override (Small Icon)**

```
<toast>
  <visual>
    <binding template="ToastGeneric">
      <text>Meeting Reminder</text>
      <text>Team standup in 5 minutes</text>
      <image placement="appLogoOverride"
        hint-crop="circle"
        src="C:\Icons\calendar.png"/>
    </binding>
  </visual>
</toast>
```

**Supported Formats:** PNG, JPG, GIF (max 200 KB)**Hero Image (Large Banner)**

```
<toast>
  <visual>
    <binding template="ToastGeneric">
      <text>Document Ready</text>
      <text>Your report has been generated</text>
      <image placement="hero" src="C:\Images\report-preview.jpg"/>
    </binding>
  </visual>
</toast>
```

**Inline Image**

```
<toast>
  <visual>
    <binding template="ToastGeneric">
```

```

        <text>Weather Alert</text>
        <text>Severe thunderstorm warning</text>
        <image src="C:\Weather\radar.png"/>
    </binding>
</visual>
</toast>

```

## Interactive Buttons

Add up to 5 buttons:

```

<toast>
  <visual>
    <binding template="ToastGeneric">
      <text>New Email from Alice</text>
      <text>RE: Project Update - Can we meet tomorrow?</text>
    </binding>
  </visual>
  <actions>
    <action content="Reply" arguments="reply:alice@company.com" />
    <action content="Mark Read" arguments="markread:12345" />
    <action content="Delete" arguments="delete:12345" />
  </actions>
</toast>

```

**Handling Button Clicks:** Requires registering a COM server or using protocol activation (advanced).

## Text Input Fields

Allow user to type response directly in toast:

```

<toast>
  <visual>
    <binding template="ToastGeneric">
      <text>Quick Reply to Alice</text>
    </binding>
  </visual>
  <actions>
    <input id="textBox" type="text" placeholderContent="Type a reply..." />
    <action content="Send" arguments="send" />
  </actions>
</toast>

```

## Progress Bar

Show dynamic progress:

```
<toast>
  <visual>
    <binding template="ToastGeneric">
      <text>Downloading File</text>
      <progress value="0.5" valueStringOverride="50%"
status="Downloading..." />
    </binding>
  </visual>
</toast>
```

### Update Progress:

```
$data = New-Object Windows.UI.Notifications.NotificationData
$data.Values["progress"] = "0.75"
$data.Values["progressStatus"] = "Almost done..."
$history.Update($data, "download-tag", "downloads", $APP_ID)
```

## Sounds

### System Sounds

```
<audio src="ms-winsoundevent:Notification.Default"/>
<audio src="ms-winsoundevent:Notification.Mail"/>
<audio src="ms-winsoundevent:Notification.Reminder"/>
<audio src="ms-winsoundevent:Notification.SMS"/>
<audio src="ms-winsoundevent:Notification.IM"/>
<audio src="ms-winsoundevent:Notification.Looping.Alarm"/>
<audio src="ms-winsoundevent:Notification.Looping.Call"/>
```

### Custom Sound

```
<audio src="file:///C:/Sounds/custom-notification.mp3"/>
```

**Supported Formats:** MP3, WMA, WAV (max 1 MB, max 30 seconds)

### Looping Sound

```
<audio src="ms-winsoundevent:Notification.Looping.Alarm" loop="true"/>
```

---

## Security Model

## Application ID Registration

When you call `CreateToastNotifier($APP_ID)`, Windows registers this App ID in:

```
HKEY_CURRENT_USER\Software\Classes\AppUserModelId\[APP_ID]
```

**Permissions Required:** None (user-level registry access)

### Security Implications:

- Any process running as the current user can send notifications under any App ID
- App IDs are **not authenticated** - two different apps can use the same App ID
- No admin rights required

## Notification Content Security

### User Trust Model:

- Notifications display content from the calling application
- Windows does **not validate or sanitize** notification text
- Applications are trusted to show accurate, non-malicious content

### Risks:

- **Spoofing:** Malicious app could impersonate another app's App ID
- **Phishing:** Notification could display misleading text/links
- **Spam:** No built-in rate limiting (app must implement)

### Mitigations:

- Use unique, namespace-qualified App IDs: `YourCompany.YourApp.Module`
- Validate all user input before showing in notifications
- Include app branding (logo) to prevent confusion
- Don't include sensitive data in notification text (use Action Center detail view)

## Privacy Considerations

### What Windows Logs:

- Notification content (stored in Action Center database)
- Timestamp of notification
- App ID that sent notification

### Location:

```
%LOCALAPPDATA%\Microsoft\Windows\Notifications\wpndatabase.db
```

### Retention:

- Up to 20 notifications per app
- Cleared when user clears Action Center
- Can be cleared by IT admin via Group Policy

### Cross-Device Sync:

- If user enables "Get notifications from apps and other senders" + "Sync across devices"
- Notifications sync to Microsoft account
- Stored on Microsoft servers temporarily
- **Don't send PII, credentials, or sensitive data in notifications**

## Code Execution Security

**Activation Arguments:** When user clicks a toast button, the `arguments` value is passed back to your app:

```
<action content="Open Document" arguments="file://C:\Docs\report.pdf"/>
```

**Security Risk:** Command injection if arguments are not validated

### Safe Pattern:

```
function Handle-NotificationActivation {
    param([string]$Arguments)

    # Validate argument format
    if ($Arguments -match '^action:(\w+):(\d+)$') {
        $action = $Matches[1]
        $id = $Matches[2]

        switch ($action) {
            "reply" { Send-Reply -MessageId $id }
            "delete" { Remove-Item -ItemId $id }
        }
    }
    else {
        Write-Warning "Invalid activation arguments: $Arguments"
    }
}
```

### Never use:

```
# DANGEROUS - allows arbitrary code execution
Invoke-Expression $Arguments
```

## Sandboxing

- Notifications run in the Windows Notification Platform service
  - Limited file system access (can load local images)
  - No network access from notification XML itself
  - Interactive elements (buttons, inputs) require activation contract
- 

## Multi-Machine Considerations

Scenario: User Works on Multiple Machines

**Question:** If you show a notification on Machine A, does it appear on Machine B?

**Answer: Depends on Windows configuration**

How Cross-Device Notifications Work

**Requirements for Sync:**

1. User signed in with Microsoft Account (not local account)
2. Settings → System → Notifications → "Get notifications from apps and other senders" = ON
3. Settings → System → Notifications → "Show me notifications from all of my devices" = ON
4. Same app installed on both machines with same App ID

**What Syncs:**

- Toast notification content (title, message)
- Notification timestamp
- App ID
- Tag and Group

**What Doesn't Sync:**

- Images (local file paths don't work across machines)
- Progress bars (state not synced)
- Custom sounds (file paths)
- Interactive button state (not synced)

Best Practices for Multi-Machine Scenarios

### 1. Use HTTP/HTTPS URLs for Images

Instead of:

```
<image src="C:\Icons\calendar.png"/>
```

Use:

```
<image src="https://yourcdn.com/icons/calendar.png"/>
```

## 2. Include Machine Context

```
$machineName = $env:COMPUTERNAME
Show-WindowsNotification `
    -Title "Document Saved" `
    -Message "Report.docx saved on $machineName"
```

## 3. Tag Notifications with Machine ID

```
$tag = "email-001-$env:COMPUTERNAME"
Show-WindowsNotification -Title "New Email" -Message "From Alice" -Tag $tag
```

This prevents conflicts if same notification sent from different machines.

## 4. Server-Side Notification Deduplication

If using a centralized system:

```
# Server tracks which machine showed notification
$notificationLog = @{
    Id = "email-001"
    ShownOnMachines = @("MACHINE-A", "MACHINE-B")
    Content = "New Email from Alice"
}

# Client queries server before showing
function Show-NotificationIfNotSeen {
    param([string]$NotificationId)

    $machine = $env:COMPUTERNAME
    $status = Get-NotificationStatus -Id $NotificationId -Machine $machine

    if (-not $status.ShownOnThisMachine) {
        Show-WindowsNotification -Title $status.Title -Message $status.Message
        Update-NotificationStatus -Id $NotificationId -Machine $machine -Shown
    }
    $true
}
```

## Notification Action Center Sync Behavior

**Important:** Action Center sync is **best-effort**:

- Syncs every 15-30 minutes (not real-time)
- Requires internet connection
- May be delayed or dropped if network unreliable
- User can disable sync anytime

**Don't rely on sync for critical notifications.** Use alternative delivery:

- Email notifications
- Server-side tracking
- Database logging

---

## PowerShell Implementation Guide

### Minimal Working Example

```
function Show-SimpleToast {
    param(
        [Parameter(Mandatory)][string]$Title,
        [Parameter(Mandatory)][string]$Message
    )

    # Load WinRT types
    [Windows.UI.Notifications.ToastNotificationManager, Windows.UI.Notifications,
    ContentType = WindowsRuntime] | Out-Null
    [Windows.UI.Notifications.ToastNotification, Windows.UI.Notifications,
    ContentType = WindowsRuntime] | Out-Null
    [Windows.Data.Xml.Dom.XmlDocument, Windows.Data.Xml.Dom.XmlDocument,
    ContentType = WindowsRuntime] | Out-Null

    # Create XML
    $template = @"
<toast>
  <visual>
    <binding template="ToastText02">
      <text id="1">$Title</text>
      <text id="2">$Message</text>
    </binding>
  </visual>
</toast>
"@

    # Show notification
    $xml = [Windows.Data.Xml.Dom.XmlDocument]::new()
    $xml.LoadXml($template)
    $toast = [Windows.UI.Notifications.ToastNotification]::new($xml)

    [Windows.UI.Notifications.ToastNotificationManager]::CreateToastNotifier("MyApp").
    Show($toast)
}
```

```
# Usage
Show-SimpleToast -Title "Test" -Message "Hello World"
```

## Production-Ready Implementation

```
function Show-WindowsNotification {
    <#
    .SYNOPSIS
        Shows a Windows toast notification with advanced options.

    .DESCRIPTION
        Displays a modern Windows notification using the WinRT Toast API.
        Supports images, sounds, duration, grouping, and logging.

    .PARAMETER Title
        Notification title (first line, bold text)

    .PARAMETER Message
        Notification message (second line, regular text)

    .PARAMETER AppId
        Application identifier for grouping. Default:
        "CreateEmailEventAndTeamsChat"

    .PARAMETER Tag
        Unique tag for this notification (enables replacement/removal)

    .PARAMETER Group
        Group identifier for related notifications

    .PARAMETER ImagePath
        Path to image file for app logo override (circle icon)

    .PARAMETER Sound
        Sound to play: Default, Mail, Reminder, SMS, IM, Silent

    .PARAMETER Duration
        How long to display: Short (5s), Long (25s), Default (7s)

    .EXAMPLE
        Show-WindowsNotification -Title "Email Sent" -Message "Email delivered
        successfully"

    .EXAMPLE
        Show-WindowsNotification -Title "Meeting Soon" -Message "Daily standup in
        5 minutes" -Sound Reminder -Duration Long
    #>

    [CmdletBinding()]
    param(
```

```

[Parameter(Mandatory)]
[string]$Title,

[Parameter(Mandatory)]
[string]$Message,

[string]$AppId = "CreateEmailEventAndTeamsChat",

[string]$Tag = $null,

[string]$Group = $null,

[ValidateScript({Test-Path $_ -PathType Leaf})]
[string]$ImagePath = $null,

[ValidateSet('Default', 'Mail', 'Reminder', 'SMS', 'IM', 'Silent')]
[string]$Sound = 'Default',

[ValidateSet('Short', 'Long', 'Default')]
[string]$Duration = 'Default'
)

try {
    # Load WinRT types
    [Windows.UI.Notifications.ToastNotificationManager,
Windows.UI.Notifications, ContentType = WindowsRuntime] | Out-Null
    [Windows.UI.Notifications.ToastNotification, Windows.UI.Notifications,
ContentType = WindowsRuntime] | Out-Null
    [Windows.Data.Xml.Dom.XmlDocument, Windows.Data.Xml.Dom.XmlDocument,
ContentType = WindowsRuntime] | Out-Null

    # Escape XML special characters
    $TitleEscaped = [System.Security.SecurityElement]::Escape($Title)
    $MessageEscaped = [System.Security.SecurityElement]::Escape($Message)

    # Build XML template
    $toastDuration = if ($Duration -ne 'Default') { "
duration=`"$($Duration.ToLower())`"" } else { "" }

    $imageXml = ""
    if ($ImagePath) {
        $ImagePathEscaped =
[System.Security.SecurityElement]::Escape($ImagePath)
        $imageXml = "<image placement=`"appLogoOverride`" hint-crop=`"circle`"
src=`"$ImagePathEscaped`"/>"
    }

    $soundSrc = switch ($Sound) {
        'Default' { "ms-winsoundevent:Notification.Default" }
        'Mail' { "ms-winsoundevent:Notification.Mail" }
        'Reminder' { "ms-winsoundevent:Notification.Reminder" }
        'SMS' { "ms-winsoundevent:Notification.SMS" }
        'IM' { "ms-winsoundevent:Notification.IM" }
        'Silent' { $null }
    }

```

```

    }

    $audioXml = if ($Sound -eq 'Silent') {
        '<audio silent="true"/>'
    } elseif ($soundSrc) {
        "<audio src=`"$soundSrc`"/>"
    } else {
        ""
    }

    $template = @"
<toast$toastDuration>
    <visual>
        <binding template="ToastGeneric">
            <text>$TitleEscaped</text>
            <text>$MessageEscaped</text>
            $imageXml
        </binding>
    </visual>
    $audioXml
</toast>
"@

    # Create notification
    $xml = [Windows.Data.Xml.Dom.XmlDocument]::new()
    $xml.LoadXml($template)
    $toast = [Windows.UI.Notifications.ToastNotification]::new($xml)

    # Set Tag and Group
    if ($Tag) { $toast.Tag = $Tag }
    if ($Group) { $toast.Group = $Group }

    # Show notification

    [Windows.UI.Notifications.ToastNotificationManager]::CreateToastNotifier($AppId).Show($toast)

    Write-Verbose "Notification shown: Title='$Title', Tag='$Tag',
Group='$Group'"
    return $true
}
catch {
    Write-Error "Failed to show notification: $($_.Exception.Message)"
    return $false
}
}

```

## Helper Functions

```

function Remove-WindowsNotification {
    param(

```

```

        [Parameter(Mandatory)][string]$Tag,
        [string]$Group = "",
        [string]$AppId = "CreateEmailEventAndTeamsChat"
    )

    $history = [Windows.UI.Notifications.ToastNotificationManager]::History
    $history.Remove($Tag, $Group, $AppId)
}

function Clear-AllWindowsNotifications {
    param([string]$AppId = "CreateEmailEventAndTeamsChat")

    $history = [Windows.UI.Notifications.ToastNotificationManager]::History
    $history.Clear($AppId)
}

function Get-WindowsNotificationHistory {
    param([string]$AppId = "CreateEmailEventAndTeamsChat")

    $history = [Windows.UI.Notifications.ToastNotificationManager]::History
    $notifications = $history.GetHistory($AppId)

    foreach ($notification in $notifications) {
        [PSCustomObject]@{
            Tag = $notification.Tag
            Group = $notification.Group
            ExpirationTime = $notification.ExpirationTime
            Content = $notification.Content.GetXml()
        }
    }
}

```

## Advanced Features

### 1. Scheduled Notifications

Deliver notification at future time:

```

function Schedule-WindowsNotification {
    param(
        [string]$Title,
        [string]$Message,
        [datetime]$DeliveryTime,
        [string]$AppId = "CreateEmailEventAndTeamsChat"
    )

    [Windows.UI.Notifications.ToastNotificationManager, Windows.UI.Notifications,
    ContentType = WindowsRuntime] | Out-Null
    [Windows.UI.Notifications.ScheduledToastNotification,
    Windows.UI.Notifications, ContentType = WindowsRuntime] | Out-Null
}

```

```
[Windows.Data.Xml.Dom.XmlDocument, Windows.Data.Xml.Dom.XmlDocument,
ContentType = WindowsRuntime] | Out-Null

$template = @"
<toast>
  <visual>
    <binding template="ToastText02">
      <text id="1">$Title</text>
      <text id="2">$Message</text>
    </binding>
  </visual>
</toast>
"@

$xml = [Windows.Data.Xml.Dom.XmlDocument]::new()
$xml.LoadXml($template)

$scheduledToast =
[Windows.UI.Notifications.ScheduledToastNotification]::new($xml, $DeliveryTime)

[Windows.UI.Notifications.ToastNotificationManager]::CreateToastNotifier($AppId).AddToSchedule($scheduledToast)
}

# Schedule notification for 30 minutes from now
Schedule-WindowsNotification -Title "Meeting Reminder" -Message "Team sync in 5 minutes" -DeliveryTime (Get-Date).AddMinutes(30)
```

## 2. Notification Update (Progress Bar)

```
function Show-ProgressNotification {
    param(
        [string]$Title,
        [double]$Progress, # 0.0 to 1.0
        [string]$Status,
        [string]$Tag = "progress",
        [string]$AppId = "CreateEmailEventAndTeamsChat"
    )

    [Windows.UI.Notifications.ToastNotificationManager, Windows.UI.Notifications,
    ContentType = WindowsRuntime] | Out-Null
    [Windows.UI.Notifications.NotificationData, Windows.UI.Notifications,
    ContentType = WindowsRuntime] | Out-Null

    $progressValue = [Math]::Round($Progress, 2)
    $progressPercent = [Math]::Round($Progress * 100)

    $template = @"
<toast>
  <visual>
    <binding template="ToastGeneric">
```

```

        <text>$Title</text>
        <progress value="$progressValue"
valueStringOverride="$progressPercent%" status="$Status" />
    </binding>
</visual>
</toast>
"@

# Show initial notification
$xml = [Windows.Data.Xml.Dom.XmlDocument]::new()
$xml.LoadXml($template)
$toast = [Windows.UI.Notifications.ToastNotification]::new($xml)
$toast.Tag = $Tag

[Windows.UI.Notifications.ToastNotificationManager]::CreateToastNotifier($AppId).Show($toast)
}

# Usage: Show progress bar
Show-ProgressNotification -Title "Downloading File" -Progress 0.0 -Status "Starting..."
Start-Sleep -Seconds 2
Show-ProgressNotification -Title "Downloading File" -Progress 0.5 -Status "50% complete"
Start-Sleep -Seconds 2
Show-ProgressNotification -Title "Downloading File" -Progress 1.0 -Status "Complete!"

```

### 3. Notification with Buttons (Advanced)

```

function Show-InteractiveNotification {
    param(
        [string]$Title,
        [string]$Message,
        [string]$AppId = "CreateEmailEventAndTeamsChat"
    )

    $template = @"
<toast launch="app-action">
    <visual>
        <binding template="ToastGeneric">
            <text>$Title</text>
            <text>$Message</text>
        </binding>
    </visual>
    <actions>
        <action content="View" arguments="action=view" />
        <action content="Dismiss" arguments="action=dismiss" />
    </actions>
</toast>
"@

```

```
$xml = [Windows.Data.Xml.Dom.XmlDocument]::new()
$xml.LoadXml($template)
$toast = [Windows.UI.Notifications.ToastNotification]::new($xml)

# Note: Handling button clicks requires COM activation or protocol handler
# See: https://learn.microsoft.com/en-us/windows/apps/design/shell/tiles-and-notifications/send-local-toast

[Windows.UI.Notifications.ToastNotificationManager]::CreateToastNotifier($AppId).Show($toast)
}
```

**Note:** Interactive notifications require additional setup:

- Register COM server for activation
- Or use protocol activation (e.g., `myapp://action=view`)

## Summary Table

| Feature                       | Capability        | PowerShell Support              |
|-------------------------------|-------------------|---------------------------------|
| Basic toast (title + message) | ✔ Yes             | ✔ Full support                  |
| Images (logo, hero, inline)   | ✔ Yes             | ✔ Full support                  |
| Custom sounds                 | ✔ Yes             | ✔ Full support                  |
| Silent notifications          | ✔ Yes             | ✔ Full support                  |
| Duration control (short/long) | ✔ Yes             | ✔ Full support                  |
| Tag and Group                 | ✔ Yes             | ✔ Full support                  |
| Replace/Remove notifications  | ✔ Yes             | ✔ Full support                  |
| Progress bar                  | ✔ Yes             | ✔ Full support                  |
| Scheduled delivery            | ✔ Yes             | ✔ Full support                  |
| Action Center history         | ✔ Yes             | ✔ Full support                  |
| Interactive buttons           | ⚠ Limited         | ⚠ Requires COM/protocol handler |
| Text input                    | ⚠ Limited         | ⚠ Requires COM/protocol handler |
| Button click handling         | ⚠ Limited         | ⚠ Requires COM/protocol handler |
| Cross-device sync             | ⚠ User-controlled | N/A (Windows feature)           |
| Security/Authentication       | ✖ No              | N/A (not supported by Windows)  |

## References

- [Windows Toast Notifications Official Docs](#)
  - [Toast Content Schema](#)
  - [Windows.UI.Notifications Namespace](#)
  - [Notification Activation \(Interactive\)](#)
- 

**Document Version:** 1.0

**Last Updated:** 2026-09-19

**For:** CreateEmailEventAndTeamsChat.Prototype.ps1 Notifications Feature